STATISTICS WITH R PROGRAMMING

ASSIGNMENT 2

Design and Develop a Modular R Programming Structure Library for a Smart Logistics and Parcel Delivery Operations System

SUBMISSION DETAILS	INFORMATION
Course	Statistics with R Programming
Course Outcome	CO2 – Loops, control structures and recursive functions in R
Assignment	Assignment 2 – Complete Worked Solution
Nature	Individual
Duration	1 Week
Total Marks	50
Student Name	P.VIGNESH
Register Number	192525013
Department / Class	BTech AIML
Submitted To	Dr.S.Radhika
Submission Date	9-11-2026

CONTENTS

- Executive Summary and Assumptions
- Task 1 – Operational Enquiry and Program-Structure Blueprint
- Task 2 – Arithmetic and Boolean Rule Engine
- Task 3 – Conditional Decision Module
- Task 4 – for(), while() and repeat Loop Processing
- Task 5 – Looping Through Non-Vector Collections
- Task 6 – Reusable Functions and Return-Value Design
- Task 7 – Complex Return Objects and Functions as Objects
- Task 8 – Recursive Logistics Algorithms
- Task 9 – Integrated Program, Testing and Refinement
- Task 10 – Efficiency and Maintainability Evaluation
- Task 11 – Changed Requirement and Revised Design
- Task 12 – Function Reference, Packaging and Recommendation
- Complete Reproducible R Script
- Conclusion and References

EXECUTIVE SUMMARY AND ASSUMPTIONS

This assignment develops a modular R program-structure library for a courier company's parcel delivery operations. The system registers parcels, calculates delivery charges, classifies service priority, checks eligibility, simulates delivery attempts, traverses hub structures, allocates work and generates operational reports. It combines arithmetic expressions, Boolean logic, if-else, switch(), for(), while(), repeat, user-defined functions, function objects and recursion.

The initial operational scale is approximately 5,000 parcels per day across 50 hubs. The design must remain usable at 500,000 parcels per day and 500 hubs. Synthetic data are used in this report. The benchmark code is provided so that actual execution time and memory observations can be collected on the student's computer; timings are not fabricated.

Assumptions

- Parcel weight is measured in kilograms and distance in kilometres.
- The base charge is ₹40, the per-kilometre rate is ₹5, and the per-kilogram rate is ₹12.
- Express service adds 25% to the pre-tax charge. Insurance adds ₹20. A 5% tax is applied after discount and insurance.
- A parcel is restricted if it is marked restricted or exceeds the permitted weight threshold of 30 kg.
- Express eligibility requires weight no more than 10 kg, distance no more than 500 km, and a non-restricted parcel.
- A parcel can have at most three delivery attempts in the simulation. A successful delivery stops further attempts.
- The hub hierarchy is represented as nested lists with a parcel count at leaf hubs.
- The assignment uses base R to avoid requiring external packages.

System overview

Module	Main R construct	Purpose
Charge engine	Arithmetic/vectorised expressions	Calculate base, distance, weight, discount, insurance and tax.
Eligibility engine	Boolean expressions	Check express, insurance and COD conditions.
Priority engine	if-else / ifelse / switch	Classify standard, express and restricted parcels.
Processing engine	for / while / repeat	Process parcel batches and delivery attempts.
Collection traversal	Lists, data frames, matrices, factors, environments	Iterate through hubs and operational records.
Function library	User-defined functions and closures	Reusable validated operations.
Hierarchy analysis	Recursion and iteration	Total parcels through nested hub networks.
Reporting	Lists and data frames	Summarise parcel counts, charges, priority and tests.

TASK 1 – FORMULATE THE OPERATIONAL ENQUIRY AND CREATE THE PROGRAM-STRUCTURE BLUEPRINT

1.1 Operational requirements

Operation	Inputs	Output	Constraints
Parcel registration	Parcel ID, weight, distance, destination, service	Validated parcel record	Unique ID; positive weight/distance; known service
Charge calculation	Weight, distance, service, insurance, discount	Numeric delivery charge	No negative values; clear tax order
Priority classification	Weight, distance, restricted flag, service	Standard / Express / Restricted	Missing values handled explicitly
Hub routing	Origin hub, destination hub, distance matrix	Route and route distance	Known hubs; valid non-negative distance
Delivery attempts	Parcel ID, attempt outcome, maximum attempts	Delivered / Failed / Returned	Finite maximum attempts
Workload allocation	Hub parcel queue and agent capacity	Agent assignment	Capacity cannot be exceeded
Daily reporting	Processed parcels and charges	Summary list/data frame	Counts and totals must reconcile

1.2 Program-structure blueprint

Stage	R structure	How it cooperates
Input validation	if-else, stopifnot(), is.na()	Reject malformed or incomplete records.
Eligibility	Relational/logical operators	Produce Boolean flags for services.
Priority	if-else and switch()	Choose the appropriate service category.
Charge calculation	Arithmetic and vectorised operations	Calculate charges for one parcel or a batch.
Batch processing	for() with seq_along()	Process a known collection of parcels.
Retries and queues	while() / repeat / break	Continue until delivered or retry limit reached.
Hub traversal	for loops, apply-family functions	Traverse lists, matrices and environments.
Hierarchy totals	Recursive functions	Aggregate parcel counts across nested hubs.
Reporting	Functions returning lists/data frames	Provide consistent outputs for management.

1.3 Reproducible R session

```
# Reproducible setup
rm(list = ls())
set.seed(2026)
options(stringsAsFactors = FALSE)
cat("R version:", R.version.string, "\n")
cat("Working directory:", getwd(), "\n")
# Base R is sufficient for this assignment.
sessionInfo()
```

1.4 Experimental enquiry questions

1. When is a for() loop more appropriate than a while() loop for parcel processing?

2. How do NA, NaN, Inf and NULL affect arithmetic and Boolean conditions?
3. When should return() be called explicitly, and when is the last evaluated expression enough?
4. How do recursive and iterative approaches compare when counting parcels in nested hub networks?
5. Does vectorised charge calculation execute faster than a parcel-by-parcel loop at larger volumes?
6. How can a retry loop guarantee termination even when a parcel is never delivered?

1.5 Acceptance criteria

Criterion	Acceptance test
Correctness	Charge and priority results match manually calculated examples.
Termination	Retry loops always stop at success or a fixed attempt limit.
Type consistency	Functions return documented numeric, logical, list or data-frame objects.
NA safety	Missing inputs are flagged rather than silently treated as TRUE/FALSE.
Reusability	Functions support default arguments and can be called from loops.
Traceability	Each parcel retains ID, status, attempts and calculated charge.
Scalability	Benchmark supports 5,000 and 500,000 parcel records.
Maintainability	Functions have clear arguments, validation and return contracts.

TASK 2 – DESIGN AN ARITHMETIC AND BOOLEAN RULE ENGINE FOR DELIVERY CHARGES AND ELIGIBILITY

2.1 Arithmetic charge formula

The delivery charge is calculated in stages. First calculate the base, distance and weight components. Then apply the express surcharge, insurance fee, discount and tax. The exact order is documented so that the same parcel always receives the same result.

Base charge = ₹40 + (distance × ₹5) + (weight × ₹12). Express surcharge = 25% of the base charge. Insurance fee = ₹20 when selected. Discount is deducted before tax. Tax = 5% of the discounted amount plus insurance.

```
calculate_charge <- function(weight, distance,
                             service = c("standard", "express"),
                             insured = FALSE,
                             discount_rate = 0) {
  service <- match.arg(service)
  if (length(weight) != 1 || is.na(weight) ||
      !is.numeric(weight) || weight <= 0)
    stop("Weight must be one positive number")
  if (length(distance) != 1 || is.na(distance) ||
      !is.numeric(distance) || distance < 0)
    stop("Distance must be one non-negative number")
  if (!is.logical(insured) || length(insured) != 1 ||
```

```
    is.na(insured))
    stop("insured must be TRUE or FALSE")
  if (length(discount_rate) != 1 || is.na(discount_rate) ||
      discount_rate < 0 || discount_rate > 1)
    stop("Discount rate must be between 0 and 1")

  base <- 40 + distance * 5 + weight * 12
  service_fee <- if (service == "express") base * 0.25 else 0
  insurance_fee <- if (insured) 20 else 0
  before_discount <- base + service_fee + insurance_fee
  discount <- before_discount * discount_rate
  taxable <- before_discount - discount
  tax <- taxable * 0.05
  total <- taxable + tax

  return(list(base = base,
              service_fee = service_fee,
              insurance_fee = insurance_fee,
              discount = discount,
              tax = tax,
              total = total))
}

calculate_charge(weight=2, distance=100,
                 service="express", insured=TRUE,
                 discount_rate=0.10)
```

2.2 Arithmetic operators demonstration

```
a <- 17
b <- 5
a + b      # addition: 22
a - b      # subtraction: 12
a * b      # multiplication: 85
a / b      # division: 3.4
a ^ 2      # exponentiation: 289
a %% b     # remainder: 2
a %/% b    # integer quotient: 3

# Integer versus double
typeof(5L)
typeof(5.0)
typeof(5L + 2L)
typeof(5L / 2L)

# Recycling: shorter vector is repeated
```

```
c(10, 20, 30, 40) + c(1, 2)
# R may warn if the longer length is not a multiple of the shorter length.
c(10, 20, 30) + c(1, 2)
```

The operators %% and %/% can help determine completed slabs and remaining units. However, boundary rules must be written carefully. For example, a parcel weighing exactly 5 kg should remain in the 0–5 kg slab if the rule says 'up to and including 5 kg'.

2.3 Two alternative weight-slab rules

```
# Alternative A: explicit conditional slabs
weight_charge_if <- function(weight) {
  if (is.na(weight) || weight <= 0) stop("Invalid weight")
  if (weight <= 1) return(12)
  else if (weight <= 5) return(30)
  else if (weight <= 10) return(55)
  else return(55 + ceiling(weight - 10) * 8)
}

# Alternative B: arithmetic for additional kg above 10
weight_charge_math <- function(weight) {
  if (is.na(weight) || weight <= 0) stop("Invalid weight")
  if (weight <= 1) return(12)
  if (weight <= 5) return(30)
  if (weight <= 10) return(55)
  55 + ceiling((weight - 10) / 1) * 8
}

sapply(c(1, 5, 10, 10.1, 12),
       weight_charge_if)
sapply(c(1, 5, 10, 10.1, 12),
       weight_charge_math)
```

The conditional version is selected for the final system because the slab boundaries are explicit and easy for operations staff to review. Arithmetic is still used for the additional kilograms above 10 kg. A pure %/% formula is possible but can be harder to read at exact boundaries.

2.4 Boolean eligibility engine

```
check_eligibility <- function(weight, distance,
                              restricted = FALSE,
                              cod_requested = FALSE,
                              insured = FALSE) {
  express_ok <- !is.na(weight) && !is.na(distance) &&
                weight <= 10 && distance <= 500 &&
                !isTRUE(restricted)
  insurance_ok <- isTRUE(insured) && weight <= 50
```

```
  cod_ok <- isTRUE(cod_requested) && distance <= 1000

  list(express_eligible = express_ok,
       insurance_eligible = insurance_ok,
       cod_eligible = cod_ok)
}

check_eligibility(3, 120, FALSE, TRUE, TRUE)

# Element-wise versus short-circuit operators
x <- c(TRUE, FALSE, NA)
y <- c(FALSE, TRUE, TRUE)
x & y       # element-wise: returns a vector
x | y
!x
xor(x, y)
any(x, na.rm=TRUE)
all(x, na.rm=TRUE)
isTRUE(NA)  # FALSE

# && and || evaluate only the first element
c(TRUE, FALSE) && c(FALSE, TRUE)
c(TRUE, FALSE) || c(FALSE, TRUE)
```

Use & and | for vector-wide comparisons. Use && and || inside scalar if conditions because they inspect only the first element. isTRUE() is useful when a condition must be exactly TRUE; NA is not accepted as true.

2.5 Special values and test cases

```
# Special values
TRUE + 1
FALSE * 10
NA + 5
NaN + 2
Inf + 10
Inf - Inf
NULL + 1
is.na(NA)
is.nan(NaN)
is.finite(Inf)

# Logical conditions involving NA
TRUE & NA
FALSE & NA
TRUE | NA
FALSE | NA
```

```
isTRUE(NA)

# NULL has length zero
length(NULL)
numeric(0) + 2

# Integer and double
typeof(4L)
typeof(4.0)
typeof(4L * 2L)
typeof(4L / 2L)
```

Value / operation	R behaviour	Design response
TRUE + 1	TRUE is coerced to numeric 1; result 2	Do not use logical flags as quantities in business calculations.
FALSE * 10	FALSE behaves as numeric 0; result 0	Keep Boolean decisions separate from charge values.
NA + 5	NA	Flag missing inputs before calculation.
NaN + 2	NaN	Reject non-finite numeric input.
Inf + 10	Inf	Use is.finite() to reject infinite distance/weight.
Inf - Inf	NaN	Check result finiteness.
NULL + 1	numeric(0)	Validate length and NULL explicitly.
TRUE & NA	NA	Do not let missing eligibility silently pass.
FALSE & NA	FALSE	A known false condition determines the AND result.
TRUE | NA	TRUE	A known true condition determines the OR result.
FALSE | NA	NA	Condition remains unknown.
c(1,2,3)+c(4,5)	Vector recycling; warning may occur	Use equal-length inputs or explicitly recycle with checks.

2.6 Input validation helper

```
validate_numeric_scalar <- function(x, name,
                                    allow_zero = TRUE) {
  if (is.null(x) || length(x) != 1 ||
      !is.numeric(x) || !is.finite(x))
    stop(name, " must be one finite numeric value")
  if (allow_zero && x < 0)
    stop(name, " cannot be negative")
  if (!allow_zero && x <= 0)
    stop(name, " must be greater than zero")
  invisible(TRUE)
}
```

TASK 3 – CONSTRUCT A CONDITIONAL DECISION MODULE USING IF-ELSE AND CONTROL STATEMENTS

3.1 Scalar if-else classifier

```
classify_parcel <- function(weight, distance,
                            restricted = FALSE) {
  if (is.na(weight) || is.na(distance) || is.na(restricted))
    return(NA_character_)
  if (isTRUE(restricted) || weight > 30)
    return("Restricted")
  else if (weight <= 10 && distance <= 500)
    return("Express")
  else
    return("Standard")
}

classify_parcel(2, 100, FALSE)
classify_parcel(20, 800, FALSE)
classify_parcel(35, 100, FALSE)
classify_parcel(NA, 100, FALSE)
```

3.2 Vectorised ifelse() version

```
classify_vector <- function(weight, distance, restricted) {
  result <- ifelse(is.na(weight) | is.na(distance) |
                     is.na(restricted), NA_character_,
            ifelse(restricted | weight > 30, "Restricted",
            ifelse(weight <= 10 & distance <= 500,
                   "Express", "Standard")))
  result
}
classify_vector(c(2,20,35,NA),
                c(100,800,100,100),
                c(FALSE,FALSE,FALSE,FALSE))
```

3.3 switch() for service selection

```
service_multiplier <- function(service) {
  switch(service,
    standard = 1.00,
    express = 1.25,
    same_day = 1.60,
    stop("Unknown service type")
}
service_multiplier("express")
service_multiplier("standard")
# service_multiplier("overnight") produces an error
```

Approach	Best use	Strength	Limitation
if-else	One parcel / scalar decision	Readable ordered rules	Not intended for vector inputs.
ifelse()	Vector of parcel records	Compact vectorised classification	Nested conditions can become hard to read; NA must be planned.
switch()	Choose one named service operation	Clear multi-way selection	Not a vectorised classifier.

3.4 break, next, stop() and warning()

```
validate_parcel_row <- function(row) {
  if (is.na(row$parcel_id) || row$parcel_id == "")
    return("missing_id")
  if (is.na(row$weight) || !is.finite(row$weight) ||
      row$weight <= 0)
    return("invalid_weight")
  "valid"
}

process_with_controls <- function(parcels) {
  output <- vector("list", nrow(parcels))
  kept <- 0L

  for (i in seq_len(nrow(parcels))) {
    status <- validate_parcel_row(parcels[i, , drop=FALSE])
    if (status == "missing_id") {
      warning("Skipping row ", i, ": missing parcel ID")
      next
    }
    if (status == "invalid_weight") {
      warning("Skipping row ", i, ": invalid weight")
      next
    }
    if (parcels$weight[i] > 100) {
      warning("Critical overweight parcel; stop this batch")
      break
    }
    kept <- kept + 1L
    output[[kept]] <- parcels[i, , drop=FALSE]
  }
  if (kept == 0L) return(data.frame())
  do.call(rbind, output[seq_len(kept)])
}
```

next skips the current iteration. break exits the loop. stop() halts a function with an error and should be used for critical invalid inputs. warning() reports a problem while allowing the workflow to continue when safe.

TASK 4 – DEVELOP LOOP-BASED PROCESSING USING for() AND while() LOOPS

4.1 Representative parcel data

```
parcels <- data.frame(
  parcel_id = c("PK001","PK002","PK003","PK004","PK005"),
  weight = c(2.5, 12, 5, 35, 1.2),
  distance = c(120, 650, 300, 80, 40),
  service = c("express","standard","express","standard","standard"),
  insured = c(TRUE,FALSE,TRUE,FALSE,FALSE),
  restricted = c(FALSE,FALSE,FALSE,TRUE,FALSE),
  stringsAsFactors = FALSE
)
parcels$priority <- mapply(classify_parcel,
                           parcels$weight, parcels$distance,
                           parcels$restricted)
parcels
```

4.2 for() loop with pre-allocation

```
process_parcel_batch <- function(data) {
  n <- nrow(data)
  charges <- numeric(n)
  priorities <- character(n)
  for (i in seq_along(data$parcel_id)) {
    priorities[i] <- classify_parcel(
      data$weight[i], data$distance[i], data$restricted[i])
    charge_result <- calculate_charge(
      data$weight[i], data$distance[i],
      service = data$service[i],
      insured = data$insured[i],
      discount_rate = 0
    )
    charges[i] <- charge_result$total
  }
  data$priority <- priorities
  data$delivery_charge <- charges
  data
}
processed_parcels <- process_parcel_batch(parcels)
processed_parcels
```

4.3 while() delivery-attempt simulation

```
simulate_delivery <- function(parcel_id,
                              success_on = 2L,
                              max_attempts = 3L) {
  attempt <- 0L
  delivered <- FALSE
  history <- character()

  while (!delivered && attempt < max_attempts) {
    attempt <- attempt + 1L
    delivered <- (attempt == success_on)
    history[attempt] <- if (delivered) "Delivered" else "Attempt failed"
  }

  status <- if (delivered) "Delivered" else "Returned to hub"
  list(parcel_id=parcel_id, attempts=attempt,
       status=status, history=history)
}
simulate_delivery("PK001", success_on=2, max_attempts=3)
simulate_delivery("PK002", success_on=9, max_attempts=3)
```

4.4 repeat loop with break

```
process_hub_queue <- function(queue, capacity) {
  processed <- character()
  while (length(queue) > 0 && length(processed) < capacity) {
    current <- queue[1]
    queue <- queue[-1]
    processed <- c(processed, current)
  }
  list(processed=processed, remaining=queue)
}

repeat_until_empty <- function(queue, max_steps=1000L) {
  processed <- character()
  steps <- 0L
  repeat {
    if (length(queue) == 0) break
    steps <- steps + 1L
    if (steps > max_steps) stop("Safety limit reached")
    processed <- c(processed, queue[1])
    queue <- queue[-1]
  }
  processed
}
repeat_until_empty(c("PK01","PK02","PK03"))
```

4.5 Pre-allocation versus growing vectors

```
# Preferred: pre-allocate output
n <- 10000
preallocated <- numeric(n)
for (i in seq_len(n)) preallocated[i] <- i * 2

# Less efficient pattern: grow an object repeatedly
growing <- numeric(0)
for (i in seq_len(n)) growing <- c(growing, i * 2)

identical(preallocated, growing)
object.size(preallocated)
object.size(growing)
```

4.6 for(), while() and vectorised comparison

```
weights <- runif(10000, 0.2, 35)

loop_start <- system.time({
  loop_result <- logical(length(weights))
  for (i in seq_along(weights))
    loop_result[i] <- weights[i] <= 10
})

while_start <- system.time({
  while_result <- logical(length(weights))
  i <- 1L
  while (i <= length(weights)) {
    while_result[i] <- weights[i] <= 10
    i <- i + 1L
  }
})

vector_start <- system.time({
  vector_result <- weights <= 10
})

stopifnot(identical(loop_result, while_result))
stopifnot(identical(loop_result, vector_result))
rbind(for_loop=loop_start, while_loop=while_start,
      vectorised=vector_start)
```

Use for() when the number of records is known. Use while() when processing continues until a condition changes, such as a successful delivery or an empty queue. Use vectorised expressions for

simple operations over entire columns. Every while/repeat loop needs an explicit termination condition or safety limit.

TASK 5 – CREATE A LOOPING MODULE FOR NON-VECTOR SETS

5.1 Hub list with nested elements

```
hub_list <- list(
  North = list(city="Chennai", hubs=list(
    list(hub_id="N01", capacity=1000L, parcels=350L),
    list(hub_id="N02", capacity=800L, parcels=420L))),
  South = list(city="Madurai", hubs=list(
    list(hub_id="S01", capacity=900L, parcels=500L))),
  West = list(city="Coimbatore", hubs=list(
    list(hub_id="W01", capacity=700L, parcels=260L)))
)

# Traverse nested list using loops
hub_totals <- data.frame(region=character(),
                         hub_id=character(),
                         parcels=integer())
for (region_name in names(hub_list)) {
  region_obj <- hub_list[[region_name]]
  for (hub in region_obj$hubs) {
    hub_totals <- rbind(hub_totals,
      data.frame(region=region_name,
                 hub_id=hub$hub_id,
                 parcels=hub$parcels))
  }
}
hub_totals
```

5.2 Data frame rows and columns

```
# Safe row traversal: use drop=FALSE to preserve data-frame shape
for (i in seq_len(nrow(parcels))) {
  one_parcel <- parcels[i, , drop=FALSE]
  cat(one_parcel$parcel_id, one_parcel$weight, "\n")
}

# Traverse columns
for (column_name in names(parcels)) {
  cat(column_name, ":", class(parcels[[column_name]])[1], "\n")
}

# Apply-family comparison
parcel_ids <- lapply(parcels, class)
```

```
column_lengths <- vapply(parcels, length, integer(1))
str(parcel_ids)
column_lengths
```

5.3 Hub distance matrix

```
hub_ids <- c("H1","H2","H3","H4")
distance_matrix <- matrix(
 c(0,10,25,40,
   10,0,18,30,
   25,18,0,12,
   40,30,12,0),
 nrow=4, byrow=TRUE,
 dimnames=list(hub_ids,hub_ids)
)

# Traverse matrix rows and columns
for (i in seq_len(nrow(distance_matrix))) {
  row_total <- sum(distance_matrix[i, ])
  cat("Total distance row", rownames(distance_matrix)[i],
      "=", row_total, "\n")
}
apply(distance_matrix, 1, sum)
apply(distance_matrix, 2, min)
```

5.4 Factor levels and environment

```
service_type <- factor(
 c("standard","express","standard","same_day","express"),
 levels=c("standard","express","same_day")
)
for (level_name in levels(service_type)) {
  cat(level_name, "count:",
      sum(service_type == level_name), "\n")
}
lapply(levels(service_type),
       function(z) sum(service_type == z))

# Environment stores mutable hub status
hub_status <- new.env(parent=emptyenv())
hub_status$H1 <- list(active=TRUE, queued=30L)
hub_status$H2 <- list(active=TRUE, queued=42L)
hub_status$H3 <- list(active=FALSE, queued=0L)

for (nm in ls(hub_status)) {
  info <- get(nm, envir=hub_status)
  cat(nm, "active:", info$active,
```

```
      "queue:", info$queued, "\n")
}
```

5.5 Apply-family versus explicit loops

Collection	Explicit loop	Apply-family alternative	Comment
List of hubs	for over names and nested hub lists	lapply(hub_list, function(x) ...)	lapply returns a list and preserves flexible results.
Data-frame columns	for over names(parcels)	lapply(parcels, class), vapply()	vapply gives a predictable output type.
Distance matrix	for over rows	apply(distance_matrix, 1, sum)	apply is concise for matrix margins.
Factor levels	for over levels()	lapply(levels(f), function(z) sum(f==z))	Explicit loop is easier for side effects or progress display.

5.6 Non-vector collection pitfalls

```
# Pitfall 1: data-frame row extraction may coerce mixed columns
mixed_df <- data.frame(id="PK01", weight=2.5,
                       insured=TRUE, stringsAsFactors=FALSE)
mixed_df[1, ]       # row may become a list-like row representation
as.character(mixed_df[1, ])  # coercion can lose original types
mixed_df[1, , drop=FALSE]    # preserve data-frame columns and types

# Pitfall 2: zero-length collections
empty_list <- list()
for (item in empty_list) {
  print(item)
}
# Use seq_along() rather than 1:length(x), which becomes c(1,0)
length(empty_list)
seq_along(empty_list)
# 1:length(empty_list) is unsafe for an empty collection

# Pitfall 3: environment names
ls(hub_status)
```

Use seq_along() for vectors and lists and seq_len(nrow(x)) for data-frame rows. Avoid 1:length(x) when x may be empty because it produces an unintended sequence. Use [[ ]] to extract one list element and [, , drop=FALSE] when a data-frame subset must remain a data frame.

TASK 6 – BUILD REUSABLE FUNCTIONS WITH DEFAULT ARGUMENTS AND WELL-DESIGNED RETURN VALUES

6.1 Function library

```
# Function 1: delivery charge with defaults
```

```
delivery_charge <- function(weight, distance,
                        service=c("standard","express","same_day"),
                        insured=FALSE, discount=0) {
  service <- match.arg(service)
  validate_numeric_scalar(weight,"weight",allow_zero=FALSE)
  validate_numeric_scalar(distance,"distance",allow_zero=TRUE)
  if (!is.logical(insured) || length(insured)!=1 || is.na(insured))
    stop("insured must be TRUE or FALSE")
  if (discount < 0 || discount > 1) stop("discount must be 0 to 1")

  base <- 40 + weight*12 + distance*5
  multiplier <- switch(service,
    standard=1,
    express=1.25,
    same_day=1.60)
  amount <- base*multiplier + if (insured) 20 else 0
  amount <- amount*(1-discount)
  total <- amount*1.05
  return(total)
}

# Function 2: priority classification
priority_label <- function(weight, distance, restricted=FALSE) {
  if (missing(weight) || missing(distance))
    stop("Weight and distance are required")
  classify_parcel(weight,distance,restricted)
}

# Function 3: attempt simulation
attempt_delivery <- function(parcel_id, success_on=2L,
                         max_attempts=3L, verbose=FALSE) {
  if (max_attempts < 1) stop("max_attempts must be at least 1")
  result <- simulate_delivery(parcel_id,success_on,max_attempts)
  if (verbose) print(result)
  result
}

# Function 4: parcel report
parcel_report <- function(data, include_rows=TRUE) {
  summary <- list(
    total_parcels=nrow(data),
    delivered=sum(data$status=="Delivered",na.rm=TRUE),
    pending=sum(data$status=="Pending",na.rm=TRUE),
    total_charges=sum(data$delivery_charge,na.rm=TRUE),
    average_charge=mean(data$delivery_charge,na.rm=TRUE)
  )
```

```
  if (include_rows) summary$records <- data
  return(summary)
}
```

6.2 Defaults, named/positional arguments, missing() and match.arg()

```
# Default argument values are used when an argument is omitted
delivery_charge(weight=2,distance=100)
delivery_charge(2,100,"express",TRUE,0.10)
delivery_charge(distance=100,weight=2,service="standard")

# match.arg selects one allowed service
delivery_charge(2,100,service="exp")

# missing() checks whether the caller supplied an argument
show_discount <- function(discount) {
  if (missing(discount)) {
    cat("No discount supplied; default policy applies\\n")
    discount <- 0
  }
  discount
}
show_discount()
show_discount(0.15)

# formals() shows defaults
formals(delivery_charge)
args(delivery_charge)
```

6.3 Return-value design

Function result	Recommended return	Reason
Charge calculation	Numeric scalar or named list	Scalar for direct use; list when component breakdown is needed.
Priority classification	Character scalar/vector	Simple label for each parcel.
Attempt simulation	Named list	Includes attempt count, status and history.
Management report	Named list or data frame	Several related totals and records must be returned together.
Validation helper	Logical or structured list	Logical for simple checks; list when reasons are required.

Use explicit return() when exiting early, when several branches return values, or when the intended result should be obvious. In a short function, the last evaluated expression is returned automatically. invisible() is useful for helper functions that update or validate an object without printing it.

```
# Implicit return: last expression is returned
```

```
square_value <- function(x) x^2
square_value(5)

# Explicit return: early exit
safe_divide <- function(a,b) {
  if (b == 0) return(NA_real_)
  return(a/b)
}

# Invisible return: useful for validation/setup
check_positive <- function(x) {
  if (any(x <= 0, na.rm=TRUE)) stop("Values must be positive")
  invisible(TRUE)
}
check_positive(c(1,2,3))
```

6.4 Input validation tests

```
delivery_charge(2,100)
delivery_charge(2,100,service="express",insured=TRUE)
try(delivery_charge(-1,100))
try(delivery_charge(2,Inf))
try(delivery_charge(2,100,service="unknown"))
try(delivery_charge(2,100,discount=1.5))
priority_label(2,100)
try(priority_label())
attempt_delivery("PK100",success_on=1,max_attempts=3)
try(attempt_delivery("PK100",max_attempts=0))
```

TASK 7 – DESIGN FUNCTIONS THAT RETURN COMPLEX OBJECTS AND TREAT FUNCTIONS AS OBJECTS

7.1 Complex return object

```
calculate_parcel_record <- function(data) {
  required <- c("parcel_id","weight","distance","service","insured")
  if (!all(required %in% names(data)))
    stop("Required columns are missing")
  n <- nrow(data)
  charge <- numeric(n)
  priority <- character(n)
  for (i in seq_len(n)) {
    charge[i] <- delivery_charge(
      data$weight[i],data$distance[i],
      service=data$service[i],insured=data$insured[i])
    priority[i] <- classify_parcel(
```

```
      data$weight[i],data$distance[i],
      if ("restricted" %in% names(data)) data$restricted[i] else FALSE)
  }
  data$delivery_charge <- charge
  data$priority <- priority
  list(
    records=data,
    total_parcels=nrow(data),
    priority_counts=table(data$priority,useNA="ifany"),
    total_charge=sum(charge),
    average_charge=mean(charge)
  )
}
parcel_result <- calculate_parcel_record(parcels)
parcel_result$records
parcel_result$priority_counts
parcel_result$total_charge
```

7.2 Functions are objects

```
# Store functions in a list
operations <- list(
  charge=delivery_charge,
  classify=classify_parcel,
  retry=simulate_delivery
)
operations$charge(2,100,service="standard")
operations$classify(2,100,FALSE)

# Pass a function as an argument
apply_rule <- function(values, rule) {
  rule(values)
}
apply_rule(c(1,2,3), function(x) x*2)

# Return a function (closure)
make_distance_rate <- function(rate) {
  function(distance) {
    if (any(distance < 0,na.rm=TRUE)) stop("Negative distance")
    distance*rate
  }
}
city_rate <- make_distance_rate(5)
remote_rate <- make_distance_rate(8)
city_rate(c(10,20))
remote_rate(c(10,20))
```

```
# Anonymous function
lapply(c(1,2,3), function(x) x^2)
```

7.3 Copy-on-modify behaviour

```
change_first <- function(x) {
  x[1] <- 999
  x
}
original <- c(10,20,30)
changed <- change_first(original)
original   # remains c(10,20,30)
changed    # c(999,20,30)

# Caller must assign returned value to keep the change
original <- change_first(original)
original
```

R uses copy-on-modify semantics for ordinary vectors and many objects. A function that modifies its local argument does not automatically update the caller's variable. Return the modified object and assign it back when predictable value semantics are desired.

7.4 Shared state using an environment

```
# Environment is a reference object: updates are shared
hub_counter <- new.env(parent=emptyenv())
hub_counter$H1 <- 0L
hub_counter$H2 <- 0L

record_parcel <- function(hub, state) {
  if (!exists(hub,envir=state,inherits=FALSE))
    stop("Unknown hub")
  state[[hub]] <- state[[hub]] + 1L
  invisible(state[[hub]])
}
record_parcel("H1",hub_counter)
record_parcel("H1",hub_counter)
record_parcel("H2",hub_counter)
as.list.environment(hub_counter)

# Return-a-copy alternative
increment_counter <- function(counts,hub) {
  if (is.null(counts[[hub]])) counts[[hub]] <- 0L
  counts[[hub]] <- counts[[hub]]+1L
  counts
}
```

```
counts <- list(H1=0L,H2=0L)
counts <- increment_counter(counts,"H1")
```

Approach	Advantage	Risk / limitation	Recommendation
Return modified copy	Clear data flow; easier to test and reproduce	Caller must assign returned value	Preferred for parcel records and reports.
Environment	Efficient shared updates; useful for live counters	Hidden state and order-dependent behaviour	Use only for deliberately shared operational counters.
<<- assignment	Can update an enclosing variable	Side effects are less visible and harder to debug	Avoid in core business functions unless clearly documented.

TASK 8 – IMPLEMENT RECURSIVE SOLUTIONS FOR HIERARCHICAL LOGISTICS PROBLEMS

8.1 Nested hub hierarchy

```
hub_network <- list(
  region="Tamil Nadu",
  children=list(
    list(city="Chennai",children=list(
      list(hub_id="CH1",parcels=120L),
      list(hub_id="CH2",parcels=80L))),
    list(city="Madurai",children=list(
      list(hub_id="MD1",parcels=65L))),
    list(city="Coimbatore",children=list(
      list(hub_id="CB1",parcels=90L),
      list(hub_id="CB2",parcels=45L)))
  )
)
```

8.2 Recursive parcel total

```
total_parcels_recursive <- function(node) {
  # Base case: a leaf hub contains a parcel count.
  if (is.list(node) && !is.null(node$parcels))
    return(as.numeric(node$parcels))

  # Recursive case: sum the totals of all child nodes.
  if (is.list(node) && !is.null(node$children)) {
    if (length(node$children) == 0) return(0)
    child_totals <- vapply(node$children,
                           total_parcels_recursive,
                           numeric(1))
    return(sum(child_totals))
  }
```

```
  # Empty or unrecognised node contributes zero.
  0
}
total_parcels_recursive(hub_network)
```

8.3 Recursive route-cost calculation

```
route_cost_recursive <- function(route, distance_matrix) {
  # Base case: zero or one hub means no travel segment.
  if (length(route) <= 1) return(0)

  from <- route[1]
  to <- route[2]
  if (!(from %in% rownames(distance_matrix)) ||
      !(to %in% colnames(distance_matrix)))
    stop("Route contains an unknown hub")

  first_leg <- distance_matrix[from,to]
  remaining_cost <- route_cost_recursive(route[-1],
                                         distance_matrix)
  first_leg + remaining_cost
}
route_cost_recursive(c("H1","H2","H3","H4"),
                     distance_matrix)
```

8.4 Iterative equivalent

```
total_parcels_iterative <- function(node) {
  total <- 0
  stack <- list(node)

  while (length(stack) > 0) {
    current <- stack[[length(stack)]]
    stack <- stack[-length(stack)]

    if (!is.null(current$parcels)) {
      total <- total + current$parcels
    } else if (!is.null(current$children) &&
               length(current$children) > 0) {
      stack <- c(stack,current$children)
    }
  }
  total
}
total_parcels_recursive(hub_network)
total_parcels_iterative(hub_network)
```

```
stopifnot(total_parcels_recursive(hub_network) ==
          total_parcels_iterative(hub_network))
```

8.5 Base cases and boundary tests

```
empty_hierarchy <- list(children=list())
single_hub <- list(hub_id="ONE",parcels=12L)
total_parcels_recursive(empty_hierarchy) # 0
total_parcels_recursive(single_hub)      # 12
total_parcels_recursive(list(parcels=5L)) # 5

# Build a nested hierarchy to test depth.
make_deep_hierarchy <- function(depth) {
  node <- list(parcels=1L)
  if (depth <= 1) return(node)
  for (i in seq_len(depth-1))
    node <- list(children=list(node))
  node
}
deep_node <- make_deep_hierarchy(100)
total_parcels_recursive(deep_node)

# Compare recursion and iteration for the sample network
system.time(total_parcels_recursive(hub_network))
system.time(total_parcels_iterative(hub_network))
```

A recursive function must have a base case and a recursive case. Without a base case, the function may call itself indefinitely until the R call stack limit is reached. The iterative stack-based method avoids deep call-stack use and is preferable for very deep or untrusted hierarchies. Recursion is clear for naturally nested hub structures of moderate depth.

TASK 9 – INTEGRATE, TEST AND REFINE THE COMPLETE R PROGRAM LIBRARY

9.1 End-to-end workflow

7. Create a raw parcel dataset with parcel ID, weight, distance, service, insurance and restriction fields.
8. Validate IDs, numeric values, finite values and service names.
9. Classify each parcel as standard, express or restricted.
10. Calculate the delivery charge for accepted parcels.
11. Simulate delivery attempts and store status and attempt history.
12. Aggregate charges and priority counts by hub or service.
13. Generate a structured daily report and retain rejected records separately.

9.2 Integrated sample workflow

```
raw_parcels <- data.frame(
 parcel_id=c("PK1001","PK1002","PK1003","PK1004","PK1005"),
 hub=c("H1","H1","H2","H3","H2"),
 weight=c(2.5,12,5,35,1.2),
 distance=c(120,650,300,80,40),
 service=c("express","standard","express","standard","standard"),
 insured=c(TRUE,FALSE,TRUE,FALSE,FALSE),
 restricted=c(FALSE,FALSE,FALSE,TRUE,FALSE),
 success_on=c(2,9,1,1,3)
)

validate_parcels <- function(x) {
 required <- c("parcel_id","hub","weight","distance",
               "service","insured","restricted")
 if (!all(required %in% names(x)))
   stop("Required parcel fields are missing")
 valid_id <- !is.na(x$parcel_id) & x$parcel_id != ""
 valid_weight <- is.finite(x$weight) & x$weight > 0
 valid_distance <- is.finite(x$distance) & x$distance >= 0
 valid_service <- x$service %in% c("standard","express","same_day")
 valid_flags <- !is.na(x$insured) & !is.na(x$restricted)
 x$valid <- valid_id & valid_weight & valid_distance &
            valid_service & valid_flags
 x
}

validated <- validate_parcels(raw_parcels)
accepted <- subset(validated,valid)
rejected <- subset(validated,!valid)

accepted$priority <- mapply(classify_parcel,
 accepted$weight,accepted$distance,accepted$restricted)
accepted$delivery_charge <- mapply(
 function(w,d,s,ins) delivery_charge(w,d,s,ins),
 accepted$weight,accepted$distance,
 accepted$service,accepted$insured
)

attempt_results <- lapply(seq_len(nrow(accepted)),function(i) {
 simulate_delivery(accepted$parcel_id[i],
                   success_on=accepted$success_on[i],
                   max_attempts=3)
})
accepted$attempts <- vapply(attempt_results,
                            function(x)x$attempts,integer(1))
```

```
accepted$status <- vapply(attempt_results,
                          function(x)x$status,character(1))

daily_summary <- list(
 total_received=nrow(validated),
 accepted=nrow(accepted),
 rejected=nrow(rejected),
 delivered=sum(accepted$status=="Delivered"),
 returned=sum(accepted$status=="Returned to hub"),
 total_charge=sum(accepted$delivery_charge),
 priority_counts=table(accepted$priority),
 by_hub=aggregate(delivery_charge~hub,
                  data=accepted,FUN=sum)
)
str(daily_summary)
```

9.3 Systematic test plan

Test ID	Input / condition	Expected result	Evidence
L01	Valid parcel record	Accepted	valid = TRUE
L02	Missing parcel ID	Rejected	valid = FALSE
L03	Negative weight	Rejected	valid = FALSE
L04	Infinite distance	Rejected	is.finite() false
L05	Unknown service	Rejected	valid = FALSE
L06	Weight exactly 10 kg	Boundary rule applied consistently	Priority function result
L07	Delivery success on attempt 2	Two attempts then Delivered	attempts = 2
L08	No success within 3 attempts	Returned to hub	attempts = 3
L09	Empty queue	Loop terminates immediately	length(queue) = 0
L10	Empty hierarchy	Recursive total is 0	result = 0
L11	Deep hierarchy	No unexpected recursion for supported depth	recursive/iterative totals match
L12	500,000 parcel benchmark	Batch completes	timing and object size recorded

9.4 Development failure and refinement log

Failure	Cause	Refinement
Infinite retry loop	No attempt limit	Add max_attempts and termination condition.
if condition returns NA	Missing logical input	Check is.na() and use isTRUE() for scalar flags.
Incorrect recycling	Unequal vector lengths	Validate equal lengths or use scalar arguments.
Unexpected data-frame row type	Mixed column types	Use drop=FALSE and column-wise access.
Recursive stack overflow	Excessively deep hierarchy	Use iterative stack traversal for deep structures.
Output changes type	Inconsistent branch return values	Document and standardise return types.

TASK 10 – CREATE AN EVIDENCE-BASED EFFICIENCY AND MAINTAINABILITY EVALUATION

10.1 Benchmark design

The experiment compares loop-based and vectorised charge calculations and recursive versus iterative hierarchy aggregation. It runs with 5,000 and 500,000 parcels. Actual elapsed time depends on processor, memory, R version and background load. Run the code and record the measured output.

```
# Generate reproducible parcel sizes
make_parcels <- function(n, seed=2026) {
  set.seed(seed)
  data.frame(
    weight=runif(n,0.5,25),
    distance=runif(n,1,1500),
    service=sample(c("standard","express"),n,TRUE),
    insured=sample(c(TRUE,FALSE),n,TRUE)
  )
}

benchmark_charges <- function(n) {
  x <- make_parcels(n)
  t_loop <- system.time({
    ans_loop <- numeric(n)
    for (i in seq_len(n)) {
      ans_loop[i] <- delivery_charge(
        x$weight[i],x$distance[i],
        service=x$service[i],insured=x$insured[i])
    }
  })
  t_vector <- system.time({
    base <- 40 + x$weight*12 + x$distance*5
    multiplier <- ifelse(x$service=="express",1.25,1)
    insurance <- ifelse(x$insured,20,0)
    ans_vector <- (base*multiplier+insurance)*1.05
  })
  stopifnot(isTRUE(all.equal(ans_loop,ans_vector)))
  data.frame(
    parcels=n,
    loop_seconds=unname(t_loop["elapsed"]),
    vector_seconds=unname(t_vector["elapsed"]),
    data_size_bytes=as.numeric(object.size(x))
  )
}

bench_results <- rbind(
  benchmark_charges(5000),
  benchmark_charges(500000)
```

```
)
print(bench_results)

# Visual comparison
barplot(t(as.matrix(bench_results[,c("loop_seconds","vector_seconds")])),
        beside=TRUE,
        names.arg=c("5,000 parcels","500,000 parcels"),
        legend.text=c("Loop","Vectorised"),
        ylab="Elapsed seconds",
        main="Charge calculation performance")
```

10.2 Recursive and iterative comparison

```
# Generate a balanced nested hierarchy with many leaves
make_hierarchy <- function(n_hubs) {
  children <- lapply(seq_len(n_hubs),function(i) {
    list(hub_id=paste0("H",i),parcels=sample(10:100,1))
  })
  list(region="Generated Region",children=children)
}
large_hierarchy <- make_hierarchy(500)

t_recursive <- system.time(
  recursive_total <- total_parcels_recursive(large_hierarchy)
)
t_iterative <- system.time(
  iterative_total <- total_parcels_iterative(large_hierarchy)
)
stopifnot(recursive_total==iterative_total)
rbind(recursive=t_recursive,iterative=t_iterative)
object.size(large_hierarchy)
```

10.3 Structural scaling table

Dataset	Parcel records	Expected data-frame rows	Evaluation focus
Initial	5,000	5,000	Daily batch processing and charge calculation.
Changed	500,000	500,000	Vectorisation, pre-allocation, memory and batch processing.

The benchmark table generated by R should be inserted into the final submission after execution. The expected design interpretation is that vectorised operations often reduce interpreter-level loop overhead for simple column calculations. Explicit loops remain useful when each record requires multiple decisions or side effects. Recursive and iterative totals should be identical; the iterative approach is safer for very deep hierarchies.

10.4 Refinement based on the experiment

- Use vectorised arithmetic for bulk charge components when the business rules can be expressed column-wise.
- Use a for() loop for sequential workflow steps that need per-parcel validation, logging or state transitions.
- Pre-allocate result vectors and lists to avoid repeated object growth.
- Use iterative stack traversal for deep or unpredictable hub hierarchies.
- Process 500,000 parcels in chunks if the complete in-memory dataset becomes too large.

TASK 11 – DEFEND THE FINAL DESIGN UNDER A CHANGED REQUIREMENT

11.1 Changed requirement

The courier company must now process 500,000 parcels per day across 500 hubs, with multi-level hub hierarchies, several service tiers and frequent real-time status updates.

11.2 Retain, modify, replace or redesign

Component	Decision	Reason
Charge formula	Retain and vectorise	Same business formula; apply to columns for bulk processing.
Priority classifier	Retain and extend	Add service tiers and maintain explicit missing-value rules.
for() processing	Modify	Use for sequential workflow, but avoid unnecessary per-cell loops.
while()/repeat	Retain with safeguards	Useful for queue processing and retry logic.
Nested hierarchy recursion	Retain for moderate depth; add iterative option	Avoid stack failures in deep networks.
Shared hub counters	Use environment or external state store	Supports live counter updates, with controlled access.
Reports	Modify	Return structured summaries and timing metrics.
Batch storage	Redesign	Chunk processing and persistent storage for high volume.

11.3 New batch-processing component

```
process_large_batch <- function(data, chunk_size=50000L) {
  if (!is.data.frame(data)) stop("data must be a data frame")
  if (chunk_size < 1) stop("chunk_size must be positive")

  n <- nrow(data)
  if (n == 0) return(data.frame())
  starts <- seq.int(1L,n,by=chunk_size)
  output <- vector("list",length(starts))

  for (j in seq_along(starts)) {
    first <- starts[j]
```

```
    last <- min(first+chunk_size-1L,n)
    chunk <- data[first:last,,drop=FALSE]

    base <- 40 + chunk$weight*12 + chunk$distance*5
    multiplier <- ifelse(chunk$service=="express",1.25,
                  ifelse(chunk$service=="same_day",1.60,1))
    insurance <- ifelse(chunk$insured,20,0)
    chunk$delivery_charge <- (base*multiplier+insurance)*1.05
    chunk$priority <- ifelse(chunk$restricted | chunk$weight>30,
                             "Restricted",
                      ifelse(chunk$weight<=10 & chunk$distance<=500,
                             "Express","Standard"))
    output[[j]] <- chunk
  }
  do.call(rbind,output)
}

# Demonstration with a small sample
batch_demo <- process_large_batch(
  data.frame(weight=c(2,12,35),distance=c(100,700,50),
             service=c("express","standard","same_day"),
             insured=c(TRUE,FALSE,FALSE),
             restricted=c(FALSE,FALSE,TRUE)),
  chunk_size=2
)
batch_demo
```

11.4 Live status update model

```
# Environment used as a simple in-memory status registry
parcel_status <- new.env(parent=emptyenv())

update_parcel_status <- function(parcel_id,new_status) {
  allowed <- c("Registered","At Hub","In Transit",
               "Out for Delivery","Delivered","Returned")
  if (!(new_status %in% allowed)) stop("Unknown status")
  parcel_status[[parcel_id]] <- list(
    status=new_status,updated_at=Sys.time())
  invisible(parcel_status[[parcel_id]])
}
update_parcel_status("PK1001","In Transit")
update_parcel_status("PK1001","Out for Delivery")
get("PK1001",envir=parcel_status)
```

11.5 Remaining limitations and future work

- An R environment is in-memory and is not a replacement for a durable multi-user database. A production system needs transactional persistence and concurrency control.
- Chunking reduces peak memory but requires careful handling of records that span chunks and consistent reporting totals.
- Vectorised logic is faster for simple operations but can become difficult to read when business rules contain many exceptions.
- Recursion is elegant for nested hierarchies but can fail for extreme depth; iterative traversal should be used when depth is not controlled.
- The sample routing model uses a distance matrix and does not implement a full shortest-path algorithm or live traffic integration.

TASK 12 – DOCUMENT, PACKAGE AND PRESENT THE FINAL LIBRARY

12.1 Function reference

Function	Arguments / defaults	Return value	Purpose / validation
calculate_charge	weight, distance, service=c('standard','express'), insured=FALSE, discount_rate=0	Named list	Charge breakdown; finite non-negative values and valid flags.
classify_parcel	weight, distance, restricted=FALSE	Character scalar	Priority; missing values return NA.
check_eligibility	weight, distance, restricted=FALSE, cod_requested=FALSE, insured=FALSE	Named logical list	Express, insurance and COD eligibility.
simulate_delivery	parcel_id, success_on=2L, max_attempts=3L	Named list	Attempt history and final status; bounded attempts.
process_parcel_batch	data	Data frame	Classifies and calculates charge for each parcel.
delivery_charge	weight, distance, service=c('standard','express','same_day'), insured=FALSE, discount=0	Numeric scalar	Reusable charge function with defaults.
priority_label	weight, distance, restricted=FALSE	Character scalar	Wrapper for parcel classification.
attempt_delivery	parcel_id, success_on=2L, max_attempts=3L, verbose=FALSE	Named list	Retry simulation with optional display.
parcel_report	data, include_rows=TRUE	Named list	Daily parcel and charge summary.
total_parcels_recursive	node	Numeric scalar	Sum parcels in nested hierarchy.
total_parcels_iterative	node	Numeric scalar	Stack-based equivalent for deep hierarchy.
process_large_batch	data, chunk_size=50000L	Data frame	Chunk-based high-volume processing.
quality_report	data	Named list	Rows, columns, missing values and duplicates.

12.2 Function inspection

```
args(delivery_charge)
formals(delivery_charge)
```

```
body(delivery_charge)
str(formals(delivery_charge))
environment(make_distance_rate)
sessionInfo()
```

12.3 Summary-report function

```
create_daily_report <- function(processed, rejected=data.frame(),
                                timings=NULL) {
  list(
    generated_at=Sys.time(),
    received=nrow(processed)+nrow(rejected),
    accepted=nrow(processed),
    rejected=nrow(rejected),
    delivered=sum(processed$status=="Delivered",na.rm=TRUE),
    returned=sum(processed$status=="Returned to hub",na.rm=TRUE),
    priority_counts=table(processed$priority,useNA="ifany"),
    total_delivery_charge=sum(processed$delivery_charge,na.rm=TRUE),
    average_delivery_charge=mean(processed$delivery_charge,na.rm=TRUE),
    timing_results=timings,
    session=R.version.string
  )
}
# Example after the integrated workflow:
# daily_report <- create_daily_report(accepted, rejected, bench_results)
# str(daily_report)
```

12.4 Reproducible package structure

Script file	Contents
00_setup.R	Clear workspace, seed, R version and session information.
01_rules.R	Arithmetic, Boolean rules and parcel eligibility.
02_decisions.R	Priority classification and control statements.
03_loops.R	Batch processing, retry simulation and queue loops.
04_collections.R	Hub lists, data frames, matrices, factors and environments.
05_functions.R	Reusable functions, defaults and return contracts.
06_recursion.R	Recursive and iterative hierarchy operations.
07_integration_tests.R	End-to-end parcel workflow and test cases.
08_benchmarks.R	5,000 / 500,000 parcel performance experiments and graphs.
09_report.R	Function reference and daily report generation.

12.5 Recommendation for operations management

Use vectorised arithmetic for simple calculations across large parcel columns, such as base charges and tax. Use if-else for a single parcel's ordered priority decision, ifelse() for vectorised classification, and

switch() for selecting one of several service-tier rules. Use for() when processing a known collection with record-level validation, logging or state changes. Use while() for queues and delivery retries that continue until a condition is met, always with a maximum-attempt safeguard. Use recursion for naturally nested hub structures of moderate depth, and an iterative stack for very deep hierarchies. Use functions with explicit validation and documented return types so that modules can be tested independently. Use environments only for intentionally shared operational state such as live hub counters.

12.6 Lessons learned

14. A loop must have a clear termination condition. A retry process without a maximum-attempt limit can run forever.

15. NA is not the same as FALSE. Missing logical values must be handled explicitly before making business decisions.

16. Functions are first-class objects in R and can be stored, passed as arguments and returned as closures. This makes reusable program libraries possible.

17. Recursion closely matches hierarchical data, but an iterative approach is safer when the hierarchy can become extremely deep.

18. Vectorisation and pre-allocation can improve efficiency, but program structure should still prioritise correctness, traceability and maintainability.

COMPLETE REPRODUCIBLE R SCRIPT

The following consolidated script gives a runnable core workflow using base R. The expanded task sections above provide additional alternative implementations and explanations. Save the code as smart_logistics_assignment2.R and run it from a fresh R session.

```
# ============================================================
# SMART LOGISTICS AND PARCEL DELIVERY OPERATIONS
# Statistics with R Programming - Assignment 2 / CO2
# ============================================================
rm(list=ls())
set.seed(2026)
options(stringsAsFactors=FALSE)
cat("R version:",R.version.string,"\n")

# A. VALIDATION AND CHARGE FUNCTIONS
validate_numeric_scalar <- function(x,name,positive=FALSE) {
 if(is.null(x)||length(x)!=1||!is.numeric(x)||!is.finite(x))
  stop(name," must be one finite numeric value")
 if(positive && x<=0) stop(name," must be positive")
 if(!positive && x<0) stop(name," cannot be negative")
 invisible(TRUE)
}

delivery_charge <- function(weight,distance,
```

```
 service=c("standard","express","same_day"),
 insured=FALSE,discount=0) {
 service<-match.arg(service)
 validate_numeric_scalar(weight,"weight",TRUE)
 validate_numeric_scalar(distance,"distance")
 if(!is.logical(insured)||length(insured)!=1||is.na(insured))
  stop("insured must be TRUE/FALSE")
 if(length(discount)!=1||is.na(discount)||discount<0||discount>1)
  stop("discount must be between 0 and 1")
 base<-40+weight*12+distance*5
 multiplier<-switch(service,standard=1,express=1.25,same_day=1.6)
 insurance<-if(insured) 20 else 0
 before_discount<-base*multiplier+insurance
 after_discount<-before_discount*(1-discount)
 total<-after_discount*1.05
 total
}

classify_parcel <- function(weight,distance,restricted=FALSE) {
 if(is.na(weight)||is.na(distance)||is.na(restricted))
  return(NA_character_)
 if(isTRUE(restricted)||weight>30) return("Restricted")
 if(weight<=10&&distance<=500) return("Express")
 "Standard"
}

# B. SAMPLE DATA
parcels<-data.frame(
 parcel_id=c("PK001","PK002","PK003","PK004","PK005"),
 hub=c("H1","H1","H2","H3","H2"),
 weight=c(2.5,12,5,35,1.2),
 distance=c(120,650,300,80,40),
 service=c("express","standard","express","standard","standard"),
 insured=c(TRUE,FALSE,TRUE,FALSE,FALSE),
 restricted=c(FALSE,FALSE,FALSE,TRUE,FALSE),
 success_on=c(2,9,1,1,3)
)

# C. LOOP-BASED PROCESSING
process_batch <- function(data) {
 n<-nrow(data)
 charges<-numeric(n)
 priority<-character(n)
 for(i in seq_along(data$parcel_id)) {
  priority[i]<-classify_parcel(data$weight[i],
                               data$distance[i],
```

```
                          data$restricted[i])
  charges[i]<-delivery_charge(data$weight[i],data$distance[i],
                          data$service[i],data$insured[i])
 }
 data$priority<-priority
 data$delivery_charge<-charges
 data
}
processed<-process_batch(parcels)

# D. DELIVERY ATTEMPT LOOP
simulate_delivery <- function(id,success_on=2L,max_attempts=3L) {
 attempt<-0L
 delivered<-FALSE
 history<-character()
 while(!delivered&&attempt<max_attempts) {
  attempt<-attempt+1L
  delivered<-attempt==success_on
  history[attempt]<-if(delivered)"Delivered" else "Attempt failed"
 }
 list(parcel_id=id,attempts=attempt,
      status=if(delivered)"Delivered" else "Returned to hub",
      history=history)
}
attempts<-lapply(seq_len(nrow(processed)),function(i)
 simulate_delivery(processed$parcel_id[i],
                   processed$success_on[i],3L))
processed$attempts<-vapply(attempts,function(x)x$attempts,integer(1))
processed$status<-vapply(attempts,function(x)x$status,character(1))

# E. NON-VECTOR COLLECTION / HUB HIERARCHY
hub_network<-list(region="Tamil Nadu",children=list(
 list(city="Chennai",children=list(
  list(hub_id="CH1",parcels=120L),
  list(hub_id="CH2",parcels=80L))),
 list(city="Madurai",children=list(
  list(hub_id="MD1",parcels=65L)))
))
total_recursive<-function(node) {
 if(!is.null(node$parcels)) return(as.numeric(node$parcels))
 if(!is.null(node$children)&&length(node$children)>0)
  return(sum(vapply(node$children,total_recursive,numeric(1))))
 0
}
total_iterative<-function(node) {
 total<-0
```

```
 stack<-list(node)
 while(length(stack)>0) {
  current<-stack[[length(stack)]]
  stack<-stack[-length(stack)]
  if(!is.null(current$parcels)) total<-total+current$parcels
  else if(!is.null(current$children))
   stack<-c(stack,current$children)
 }
 total
}
stopifnot(total_recursive(hub_network)==total_iterative(hub_network))

# F. FUNCTION OBJECTS AND ENVIRONMENT STATE
operations<-list(charge=delivery_charge,classify=classify_parcel)
operations$charge(2,100)
counter<-new.env(parent=emptyenv())
counter$H1<-0L
increment_hub<-function(hub,state) {
 state[[hub]]<-state[[hub]]+1L
 invisible(state[[hub]])
}
increment_hub("H1",counter)
increment_hub("H1",counter)

# G. DAILY REPORT
daily_report<-list(
 received=nrow(processed),
 delivered=sum(processed$status=="Delivered"),
 returned=sum(processed$status=="Returned to hub"),
 total_charge=sum(processed$delivery_charge),
 average_charge=mean(processed$delivery_charge),
 priority_counts=table(processed$priority),
 hub_parcels=total_recursive(hub_network),
 session=R.version.string
)
str(daily_report)
print(processed)

# H. SCALABILITY BENCHMARK
make_parcels<-function(n,seed=2026) {
 set.seed(seed)
 data.frame(weight=runif(n,.5,25),
            distance=runif(n,1,1500),
            service=sample(c("standard","express"),n,TRUE),
            insured=sample(c(TRUE,FALSE),n,TRUE))
}
```

```
benchmark_charge<-function(n) {
 x<-make_parcels(n)
 t_loop<-system.time({
  loop<-numeric(n)
  for(i in seq_len(n))
   loop[i]<-delivery_charge(x$weight[i],x$distance[i],
                            x$service[i],x$insured[i])
 })
 t_vec<-system.time({
  base<-40+x$weight*12+x$distance*5
  mult<-ifelse(x$service=="express",1.25,1)
  insurance<-ifelse(x$insured,20,0)
  vec<-(base*mult+insurance)*1.05
 })
 stopifnot(isTRUE(all.equal(loop,vec)))
 data.frame(parcels=n,loop_seconds=unname(t_loop["elapsed"]),
            vector_seconds=unname(t_vec["elapsed"]),
            size_bytes=as.numeric(object.size(x)))
}
benchmark_results<-rbind(benchmark_charge(5000),
                         benchmark_charge(500000))
print(benchmark_results)
sessionInfo()
# END OF REPRODUCIBLE SCRIPT
```

CONCLUSION

The assignment demonstrates how R program structures can be combined to support a realistic parcel delivery workflow. Arithmetic expressions calculate delivery charges, Boolean expressions decide service eligibility, if-else and switch() implement business decisions, for() processes known batches, while()/repeat manage queues and delivery attempts, and recursion aggregates data through nested hub structures. User-defined functions provide reusable operations with documented defaults and return values. Functions can also be stored, passed and returned as objects.

For the changed requirement of 500,000 parcels per day, the recommended architecture combines vectorised calculations, pre-allocated results, chunk processing, bounded retry loops and iterative hierarchy traversal where depth may be large. The design preserves validation, traceability and clear return contracts while allowing the system to scale.

REFERENCES

19. R Core Team. R: A Language and Environment for Statistical Computing. R Foundation for Statistical Computing. https://www.r-project.org/

20. R Documentation. Control Flow – if, for, while, repeat, break and next. https://stat.ethz.ch/R-manual/R-devel/library/base/html/Control.html

21. R Documentation. Arithmetic Operators. https://stat.ethz.ch/R-manual/R-devel/library/base/html/Arithmetic.html

22. R Documentation. Logical Operators. https://stat.ethz.ch/R-manual/R-devel/library/base/html/Logic.html

23. R Documentation. switch – Select One of a List of Alternatives. https://stat.ethz.ch/R-manual/R-devel/library/base/html/switch.html

24. R Documentation. apply – Apply Functions Over Array Margins. https://stat.ethz.ch/R-manual/R-devel/library/base/html/apply.html

25. R Documentation. lapply – Apply a Function Over a List or Vector. https://stat.ethz.ch/R-manual/R-devel/library/base/html/lapply.html

26. R Documentation. function – Create a Function. https://stat.ethz.ch/R-manual/R-devel/library/base/html/function.html

27. R Documentation. environment – Environment Objects. https://stat.ethz.ch/R-manual/R-devel/library/base/html/environment.html

28. R Documentation. Recall – Recursive Function Calls. https://stat.ethz.ch/R-manual/R-devel/library/base/html/Recall.html

29. R Documentation. system.time – CPU Time Used. https://stat.ethz.ch/R-manual/R-devel/library/base/html/system.time.html